CSIT5760

Blockchain and Smart Contracts

Individual Project Guidelines

Project proposal deadline: 10 November 2026, 11:59 p.m. HKT

Final project deadline: 30 November 2026, 11:59 p.m. HKT

Academic Year 2026–2027

Contents

1 Project Overview **3**

2 Important Deadlines **3**

3 Project Proposal **4**

3.1 Project Title 4

3.2 Background and Motivation 4

3.3 Project Objectives 5

3.4 Proposed System and Scope 5

3.5 Preliminary Smart Contract Design 6

3.6 Technology Stack 6

3.7 Security Considerations 7

3.8 Testing Plan 7

3.9 Work Plan 8

4 Final Project Requirements **8**

4.1 Smart Contract Requirements 8

4.2 Front-End Requirement 9

5 Testnet Deployment **9**

6 Contract Verification **10**

7 Testing Requirements **10**

7.1 Positive Testing 10

7.2 Negative Testing 10

7.3 Boundary and State Testing 10

7.4 Test-Case Documentation 11

8 Smart Contract Security Analysis **11**

8.1 Manual Security Review 12

9 Final Project Report **12**

10 README Requirements **13**

11 Final Submission Package **14**

12 Sensitive Information **15**

13 Academic Integrity and Use of AI **15**

14 Blockchain and IPFS Privacy **15**

15 Code Reuse and Third-Party Libraries **16**

16 What Makes a Strong Project? **16**

17 Final Submission Checklist **16**

1 Project Overview

The individual project is intended to demonstrate your ability to design, implement, test, deploy, and evaluate a blockchain-based application using Solidity and smart contracts.

You should identify a meaningful real-world problem and develop an appropriate blockchain-based solution. Your project should demonstrate, where applicable:

- smart contract design and implementation;
- appropriate use of blockchain technology;
- Solidity programming;
- access control and authorization;
- event-driven contract interaction;
- secure handling of Ether, tokens, or NFTs;
- positive, negative, and boundary testing;
- smart contract security analysis;
- deployment to a public test network;
- source-code verification; and
- clear technical documentation.

The objective is not simply to write a Solidity contract. You should demonstrate that you understand the design decisions, security considerations, testing requirements, and operational behaviour of the system you develop.

2 Important Deadlines

Submission	**Deadline**
Project proposal	**10 November 2026, 11:59 p.m. HKT**
Final project submission	**30 November 2026, 11:59 p.m. HKT**

All deadlines use Hong Kong Time. Late submissions will be handled according to the published course policy.

3 Project Proposal

The first deliverable is a short proposal describing the project you intend to develop. The recommended length is approximately **2–3 pages**, excluding references. You may use either the supplied Microsoft Word or LaTeX proposal template.

3.1 Project Title

Provide a concise and descriptive title. Suitable project areas may include:

- decentralized charity management;
- blockchain-based rental agreements;
- NFT-based certificate verification;
- decentralized auctions;
- crowdfunding;
- tokenized reward systems;
- supply-chain verification;
- decentralized voting;
- escrow systems; and
- blockchain-based ownership or authenticity verification.

These are examples only. You are encouraged to develop your own idea.

3.2 Background and Motivation

Explain:

- the real-world problem;
- who experiences the problem;
- the intended users and stakeholders;
- limitations of existing approaches; and
- why blockchain or smart contracts are appropriate.

Blockchain should provide a meaningful benefit, such as transparency, tamper-resistant records, reduced reliance on a central authority, programmable ownership, automatic enforcement of rules, or independently verifiable transactions. If a conventional database would clearly be sufficient, you should reconsider the design or provide a convincing justification.

3.3 Project Objectives

Define clear and measurable objectives. Avoid objectives that are too general, such as "build a blockchain application."

A more suitable objective would be:

> Develop a Solidity smart contract that allows authorized issuers to create verifiable digital certificates and enables any user to verify a certificate's validity.

3.4 Proposed System and Scope

Describe what the system will and will not do. Consider:

- user roles and permissions;
- principal smart contract functions;
- data stored on-chain and off-chain;
- access-control rules;
- important events;
- handling of Ether, ERC-20 tokens, or ERC-721 NFTs;
- external services and dependencies;
- assumptions; and
- functionality deliberately excluded from the project.

The scope should be realistic for the available project period.

3.5 Preliminary Smart Contract Design

Provide a preliminary design covering, where applicable:

- contract names;
- state variables;
- mappings;
- structs;
- enums;
- modifiers and roles;
- events and custom errors; and
- major functions.

You may include an architecture, workflow, sequence, or state-transition diagram. Complete Solidity code is not required at the proposal stage.

3.6 Technology Stack

Identify the technologies you expect to use. A typical project might include:

- Solidity 0.8.x;
- Remix or Hardhat;
- MetaMask;
- the Sepolia test network;
- OpenZeppelin Contracts;
- Etherscan;
- IPFS and Pinata, where appropriate; and
- Slither or another security-analysis tool.

Clearly identify important third-party libraries and versions.

3.7 Security Considerations

Identify at least **three security risks relevant to your project** and explain how you intend to address them. Relevant risks may include:

- unauthorized access;
- reentrancy;
- double withdrawal;
- duplicate execution;
- incorrect state transitions;
- insufficient input validation;
- denial of service;
- front-running;
- timestamp dependence;
- insecure Ether transfer;
- token or NFT ownership issues; and
- exposure of sensitive information on-chain.

Do not simply list vulnerabilities. Explain why each issue is relevant to your system and how the proposed controls will be validated.

3.8 Testing Plan

Explain how you intend to validate the contract. Include:

- successful operations;
- invalid inputs;
- unauthorized calls;
- repeated operations;
- boundary conditions;
- expected reverts;
- state changes;
- event emission; and
- Ether, token, or NFT balance and ownership changes where applicable.

3.9 Work Plan

Provide a realistic schedule covering:

1. system design;
2. smart contract development;
3. unit testing;
4. security review;
5. remediation and retesting;
6. public testnet deployment;
7. Etherscan verification; and
8. final report preparation.

4 Final Project Requirements

The final submission should demonstrate a complete development lifecycle from design through implementation, testing, security analysis, deployment, verification, and documentation.

4.1 Smart Contract Requirements

Projects will differ, so not every technical feature is mandatory for every project. However, you should:

- use Solidity 0.8.x;
- define clear contract responsibilities;
- implement appropriate access control;
- validate function inputs;
- prevent unauthorized and inappropriate repeated operations;
- emit events for significant state changes;
- handle Ether securely where applicable;
- implement valid state transitions;

- document important functions;
- identify external libraries; and
- ensure that submitted code corresponds to deployed and verified code.

Where appropriate, use established OpenZeppelin implementations such as `Ownable`, `AccessControl`, `ReentrancyGuard`, `Pausable`, ERC-20, and ERC-721. Using a library does not remove the requirement to understand and explain how it works within your system.

4.2 Front-End Requirement

A sophisticated front end is not required unless it is important to the proposed project. Contract interaction may be demonstrated using Remix, deployment or interaction scripts, or an appropriate web interface.

Assessment should focus primarily on smart contract design, correctness, testing, security, deployment, verification, and demonstrated understanding. A visually sophisticated interface should not compensate for weak smart contract implementation or inadequate testing.

5 Testnet Deployment

The completed contract should be deployed to an approved public Ethereum test network, normally **Sepolia**. Record:

- network name and chain ID;
- contract name and address;
- deployer address;
- deployment transaction hash and block number;
- compiler version;
- optimization configuration and run count;
- selected EVM version, if relevant;
- deployment date and time;
- deployment tool; and
- direct Etherscan URL.

If multiple contracts are deployed, provide this information for every relevant contract. Do not use real cryptocurrency where a test network is sufficient.

6 Contract Verification

The deployed contract should be verified on Etherscan. Verification settings must match the deployment settings, including:

- compiler version;
- optimization setting and run count;
- EVM version where relevant;
- constructor arguments; and
- external library information where applicable.

The final report should include a direct link to the verified contract and appropriate verification evidence.

7 Testing Requirements

Testing is a major component of the project. Each significant requirement should have corresponding tests.

7.1 Positive Testing

Confirm that valid operations succeed. For example, an authorized certificate issuer successfully creates a certificate.

7.2 Negative Testing

Confirm that invalid or unauthorized operations fail correctly. For example, a non-authorized account attempts to issue a certificate and the transaction reverts.

7.3 Boundary and State Testing

Test system limits and state transitions, including where relevant:

- zero or empty values;
- insufficient payment or token balance;
- operations before or after a deadline;

- repeated operations;
- invalid contract states;
- maximum or minimum values;
- token balance boundaries; and
- ownership changes.

7.4 Test-Case Documentation

Important test cases should record:

Field	Required information
Test ID	Unique identifier
Purpose	Requirement or behaviour being validated
Preconditions	Required account, balance, role, or contract state
Input or steps	Operation and parameters being tested
Expected result	Intended output, state change, event, or revert
Actual result	Observed result
Status	Pass or fail

Screenshots alone are not sufficient. Explain what each item of evidence demonstrates and why the observed result is correct.

8 Smart Contract Security Analysis

A security review is required. You may use tools such as Slither, Remix Static Analysis, Mythril, or another appropriate analyzer, together with manual code review. Record tool versions and commands where possible.

For each significant finding, document:

- finding ID;
- issue title and description;
- severity;
- affected contract, function, or source-code line;
- potential impact;
- remediation;

- current status; and
- evidence of retesting.

A tool-generated report by itself is not sufficient. You must interpret the results. If a reported issue is not applicable, explain why. If an issue is fixed, retest the revised contract before claiming that it has been resolved.

8.1 Manual Security Review

Discuss protections relevant to your contract, including where applicable:

- checks-effects-interactions;
- reentrancy protection;
- access control;
- input validation;
- prevention of repeated execution;
- secure Ether handling;
- state-transition validation;
- denial-of-service considerations;
- timestamp limitations;
- token or NFT safety;
- integer arithmetic;
- on-chain privacy; and
- emergency controls.

You are not expected to claim that a contract is completely secure. Explain what was reviewed, what issues were identified, what controls were applied, what was retested, and what limitations remain.

9 Final Project Report

Use the supplied Final Project Report Template in Word or LaTeX format. The report should contain:

Chapter 1: Project Overview

Project title, objectives, background and motivation, scope, assumptions, and limitations.

Chapter 2: System Design

Architecture overview, user roles and access control, data and process design, and technology stack.

Chapter 3: Smart Contract Details

Contract functionality, interface description, important implementation decisions, and representative code excerpts.

Chapter 4: Deployment on Testnet

Deployment configuration, contract addresses, transaction hashes, block numbers, and Etherscan verification.

Chapter 5: Test Cases and Validation

Testing environment, test cases, expected and actual results, status, and labelled evidence.

Chapter 6: Smart Contract Security Audit

Audit scope, methodology, tool-based analysis, findings, remediation, retesting, and manual security review.

Chapter 7: Conclusion and Future Work

Achievements, results, limitations, lessons learned, and realistic future improvements.

The report should also include references and appropriate appendices containing complete source code, scripts, test code and output, security-analysis output, Etherscan evidence, metadata examples, and additional diagrams where relevant.

10 README Requirements

The submitted `README.md` should allow a new user to understand, compile, test, deploy, and interact with the project. Include:

1. project title and summary;
2. student name and student ID;
3. software prerequisites;
4. installation instructions;
5. compilation instructions;
6. test instructions and commands;
7. deployment instructions;
8. deployed contract addresses;
9. verified Etherscan URLs;
10. external libraries and exact versions;
11. role and account assumptions;
12. instructions for interacting with the contract;
13. known limitations; and
14. security considerations.

11 Final Submission Package

Submit one compressed file using the following naming convention:

```
StudentID_FullName_CSIT5760_FinalProject.zip
```

Recommended structure:

```
StudentID_FullName_CSIT5760_FinalProject/
|-- report.pdf
|-- README.md
|-- contracts/
|   `-- YourContract.sol
|-- tests/
|   `-- contract-tests
|-- deployment/
|   |-- deployment-information.txt
|   `-- deployment-script
|-- audit/
```

```
|   `-- security-report
`-- screenshots/
    |-- deployment.png
    |-- verification.png
    `-- testing.png
```

Test and deployment filename extensions may depend on the selected development environment.

12 Sensitive Information

> **Important:** Never submit wallet seed phrases, private keys, passwords, real API keys, access tokens, environment files containing credentials, or other authentication secrets.

Avoid unnecessary dependency directories such as `node_modules`.

13 Academic Integrity and Use of AI

The submitted project must demonstrate your own understanding and work. Any external material should be acknowledged, including source code, documentation, tutorials, academic literature, libraries, online examples, and AI-assisted material where required by course policy.

Using generative AI does not remove your responsibility for the correctness, security, originality, and explainability of the submission. You must be able to explain the code and technical decisions in your project. Students may be asked to explain or demonstrate important parts of their implementation.

14 Blockchain and IPFS Privacy

Information placed on a public blockchain may be permanently accessible. Files uploaded to IPFS may remain retrievable even if a user interface or gateway link is removed.

Do not publish confidential information, private personal information, credentials, copyrighted material without permission, or sensitive user data. Use synthetic or non-sensitive data for demonstrations wherever possible.

15 Code Reuse and Third-Party Libraries

Using established libraries such as OpenZeppelin is permitted and encouraged where appropriate. However, using a library is different from claiming that you developed that functionality yourself.

Clearly acknowledge reused or adapted code and be able to explain:

- why the library was selected;
- which functionality is used;
- how it affects the contract design; and
- relevant security assumptions and limitations.

16 What Makes a Strong Project?

A strong project does not necessarily need to be large. A focused project will generally be stronger when it demonstrates:

Clear problem → appropriate blockchain design → correct implementation → comprehensive testing → security analysis → successful deployment → verification → clear documentation

A carefully implemented escrow contract with strong authorization, valid state transitions, comprehensive tests, security analysis, and verified Sepolia deployment may demonstrate greater technical understanding than a large application containing many incomplete features.

Focus on quality, correctness, security, and demonstrated understanding rather than project size.

17 Final Submission Checklist

☐ The correct student name and ID are shown.

☐ The final report is provided as a PDF.

☐ All Solidity source files are included.

☐ Test files are included.

☐ Deployment scripts and information are included.

☐ The project compiles successfully.

☐ Tests execute successfully.

☐ Positive, negative, and boundary cases are tested.

☐ The contract is deployed to the approved test network.

☐ The deployed source is verified on Etherscan.

☐ Contract addresses and transaction hashes are correct.

☐ Security analysis has been completed.

☐ Findings, remediation, and retesting are documented.

☐ Third-party libraries and reused code are acknowledged.

☐ The README contains reproducible instructions.

☐ No private key, seed phrase, API secret, or credential is included.

☐ The compressed file follows the required naming convention.

Important: You are not expected to build the largest possible blockchain application. You are expected to demonstrate that you can design, implement, test, secure, deploy, verify, document, and explain a well-scoped smart contract system.

End of Individual Project Guidelines